

Mashburn Analytics

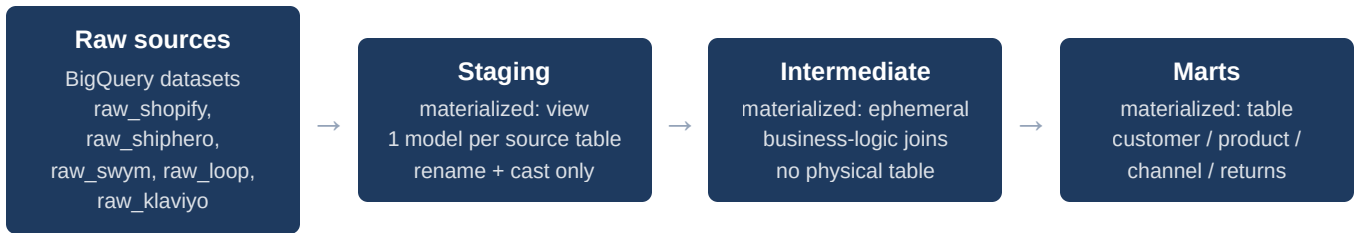
dbt + BigQuery portfolio project — interview prep sheet · Data Engineer role

Elevator pitch: A mock dbt project modeled after Sid Mashburn's actual data stack — Shopify, ShipHero, Swym, Loop Returns, and Klaviyo — built to close the BigQuery platform gap before the next round and to prove I can reason about specialty-retail / DTC data problems, not just write SQL. I designed the full source-to-mart architecture, wrote realistic mock data generators for all five sources, and scaffolded every staging/intermediate/mart model with the exact transformation logic documented as TODOs — the plan is complete and implementation is in progress.

1. Why this project exists

- **Closes a real gap:** prior BigQuery experience was limited — this is deliberate reps on BigQuery-specific SQL (nested/repeated fields, partitioning, clustering) before round 2.
- **Mirrors the actual company stack:** instead of a generic dbt demo, every source table matches what Mashburn plausibly runs in production (Shopify + Loop + Swym + Klaviyo is a very common specialty-retail DTC stack).
- **Proves domain thinking:** the business questions (below) are retail-specific — sell-through, wholesale vs. DTC channel split, wishlist demand signals, return economics — not generic "total sales by month."

2. Architecture — medallion pattern



Mock data flows in via Python ingestion scripts (`generate_seed.py` , `load_to_bigquery.py`) so the whole pipeline runs without any live Mashburn data — full local/dev loop with a free-tier BigQuery project.

3. Data sources

Source	What it is	Key tables
Shopify	Orders, customers, products, inventory — core commerce	orders, order_line_items, customers, products, product_variants, inventory_levels, locations
ShipHero	WMS / fulfillment	shipments, shipping_labels (nested carrier/tracking data)
Swym	Wishlist + back-in-stock waitlist (confirmed live on shopmashburn.com — heart icon on PDPs)	wishlist_events, waitlist_signups

Loop Returns	Self-serve returns portal — dominant platform in Shopify/DTC fashion	return_requests, return_line_items
Klaviyo	Email/SMS marketing (near-universal for Shopify)	campaigns, email_events

Field names deliberately match each vendor's *real* API/export schema (e.g. Swym's `empi` / `epi` / `cts` , Loop's `outcome` enum of reject/donate/review/keep) — this is what makes the staging layer non-trivial: standardizing genuinely inconsistent vendor conventions, not just renaming clean columns.

4. Layering conventions (what I'd say if asked "walk me through your dbt structure")

Layer	Rule
staging/	One model per source table. Naming: <code>stg_<source>__<table></code> . Only renaming to snake_case, type casting, timestamp casting. No filtering, no joins, no business logic. Materialized as views (cheap, always fresh).
intermediate/	Business-logic joins across staging models (e.g. enrich order line items with product + customer context). Materialized ephemeral — compiled inline into downstream models, no physical footprint, keeps the dataset uncluttered.
marts/	Final consumption-layer tables, grouped by business domain (<code>customer</code> , <code>product</code> , <code>channel</code> , <code>returns</code>) rather than by source — this is the dimensional-modeling principle that marts should mirror how the business asks questions, not how data arrives.

5. Business questions this is designed to answer

- 1 Collection performance** — sell-through rate for editorial collections (e.g. "Ann Loves Lately") vs. standard inventory, by tag.
- 2 Customer LTV & retention** — repeat purchase rate at 90/180/365 days; which acquisition channel produces the highest-LTV customers.
- 3 Channel split** — eCommerce vs. in-store revenue share by location; which stores overperform.
- 4 Wishlist-to-purchase conversion** (Swym) — which products get wishlisted a lot but bought rarely (reorder demand signal).
- 5 Back-in-stock conversion** (Swym waitlist) — % of waitlist signups that convert after restock notification.
- 6 Return rate by product** — which SKUs/categories return most, and why.
- 7 Exchange vs. refund rate** (Loop) — exchanges retain revenue; refunds don't.
- 8 Email attribution** (Klaviyo) — revenue attributable to campaigns vs. automated flows.
- 9 Discount impact** — do discount-code customers have worse LTV? Are we training people to wait for sales?
- 10 Inventory health** — weeks of supply by SKU/location; flag aged inventory before markdown.

6. BigQuery-specific things to have cold

```

-- Partitioning & clustering: declared in dbt config, not DDL
{{ config(
    partition_by={"field": "order_date", "data_type": "date"},
    cluster_by=["channel", "location_id"]
) }}

-- Always filter on the partition column (cost + performance)
WHERE DATE(created_at) >= '2024-01-01'      -- prunes partitions
WHERE EXTRACT(YEAR FROM created_at) = 2024  -- BAD: full table scan

-- Shopify line_items arrive as ARRAY<STRUCT> (nested/repeated)
SELECT o.id, li.sku, li.quantity
FROM raw_shopify.orders o, UNNEST(o.line_items) AS li

-- BigQuery-native functions worth using deliberately
SAFE_DIVIDE(a, b)          -- NULL instead of divide-by-zero error
COUNTIF(condition)       -- cleaner than COUNT(CASE WHEN ...)
ARRAY_AGG(x ORDER BY y LIMIT 1) -- e.g. first-touch attribution channel

```

The Postgres → BigQuery gap is mostly: nested/repeated fields (`UNNEST`), partition-aware filtering, and materialization strategy (views are nearly free in BQ, so staging-as-view is the default — unlike Postgres where you might lean on materialized views more sparingly).

7. Current build status — be upfront about this

Piece	Status	Notes
Repo & directory structure	Built	Full dbt project scaffold, medallion layout
Source docs (<code>sources.yml</code>)	Built	All 5 sources, real vendor field names, uniqueness/not-null tests declared
Mock data generators	Built	<code>generate_seed.py</code> (full mock dataset) + <code>generate_daily.py</code> (simulated incremental loads) + <code>load_to_bigquery.py</code>
Staging models	Scaffolded	Every file exists with source ref + exact transform logic commented in as TODOs — implementation is mechanical from here
Intermediate models	Scaffolded	Join logic specified (e.g. wishlist → purchase match on variant_id + email + date ordering)
Mart models	Scaffolded	Exact target column lists documented per mart
BigQuery project connection	Not yet	Free-tier project + <code>dbt seed / dbt run</code> is the next step

How to frame this in the interview: "I built the full design first — sources, business logic, and the exact target schema for every model are documented — and I'm implementing it top-down. I'd rather show you I can architect the whole pipeline and reason about the business logic correctly than rush half-working SQL." This is a *strength* to state plainly, not a gap to hide.

Example: what a finished mart looks like (design spec already written)

```
-- models/marts/customer/customer_ltv.sql (target design)
-- One row per customer:
--   customer_id, email, first_name, last_name, tags, city, state
--   lifetime_orders, lifetime_revenue, avg_order_value
--   first_order_at, last_order_at, days_since_last_order
--   customer_tier      (derived from tags or spend threshold)
--   acquisition_channel (source_name of the FIRST order)
-- Key question: which acquisition channel produces the highest-LTV customers?
```

If asked to whiteboard this: staging orders (1 row/order) → `int_customer_order_history` (aggregate to 1 row/customer with order counts, revenue sum, min/max dates) → join to `stg_shopify_customers` for identity attributes → derive tier + acquisition channel in the final mart `SELECT` .

8. Likely interview questions this project sets you up for

- "Walk me through your dbt project structure." → Use the layering table in Section 4.
- "How would you handle a Shopify order with nested line items in BigQuery?" → `UNNEST` example in Section 6; note staging keeps line items nested and a separate `stg_shopify_order_line_items` model does the unnest so downstream consumers choose grain.

- **"How do you decide what's a staging model vs. an intermediate model vs. a mart?"** → Staging = 1:1 with source, no logic. Intermediate = joins/business rules, ephemeral. Marts = final grain, keyed to a business question, materialized as tables for BI tool performance.
- **"What would you test?"** → `unique / not_null` on primary keys (already declared in `sources.yml`), source freshness on `created_at`, and relationship tests between marts and staging grain.
- **"Why ephemeral for intermediate models?"** → No BI tool or analyst queries them directly — keeping them out of the warehouse as physical objects avoids schema clutter and cost, since dbt inlines them into the mart's compiled SQL.
- **"What's the hardest part of this so far?"** → Standardizing genuinely inconsistent vendor field names (Swym's abbreviated `epi / cts` keys, Loop's non-obvious `outcome` enum) into a coherent staging layer — that's real integration work, not toy data.